

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

Siddharth Arya (1BM23CS328)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug-2025 to Jan-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ Bio Inspired Systems (23CS5BSBIS)” carried out by **Siddharth Arya (1BM23CS328)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Rohith Vaidya K Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	29/8/25	Genetic Algorithm for Optimization Problems	4
2	12/9/25	Particle Swarm Optimization for Function Optimization	8
3	10/10/25	Ant Colony Optimization for the Traveling Salesman Problem	12
4	17/10/25	Cuckoo Search (CS)	16
5	17/10/25	Grey Wolf Optimizer (GWO)	20
6	7/11/25	Parallel Cellular Algorithms and Programs	24
7	29/8/25	Optimization via Gene Expression Algorithms	27

Github Link:

<https://github.com/Sid-CS328/BioInspiredSystems>

Program 1

Determine the optimal allocation of investments across four assets such that the portfolio's expected return is maximized while penalizing risk and encouraging diversification. The solution should identify a set of portfolio weights that sum to one and provide the best balance between return, risk, and diversification based on the given return values and covariance matrix.

Algorithm:

DATE: 29/3/25 PAGE:

Lab - 1

★ Genetic Algorithm

5 main phases:

- 1) Initialization
- 2) Fitness Assignment
- 3) Selection
- 4) Crossover
- 5) Termination

Steps:

1) Select encoding Techniques
0 to 31

2) Select the initial population - "4"

SNo.	Initial population	X value	Fitness $f(x)=x^2$	Probability $\frac{f(x)}{\sum f(x)}$	Probability %	Expected count $\frac{f(x)}{\text{Avg}(\sum f(x))}$	Actual count (Roulette wheel)
1	01100	12	144	0.1247	12.47	0.49	1
2	11001	25	625	0.5411	54.11	2.164	2
3	00101	5	25	0.0216	2.16	0.086	0
4	10011	19	361	0.3125	31.25	1.25	1
Sum			1155				
Avg			288.75				
Max			625				

Code:

```
import random
import numpy as np
```

```
expected_returns = np.array([0.12, 0.10, 0.15, 0.09])
```

```
cov_matrix = np.array([
    [0.006, -0.002, 0.004, 0.000],
    [-0.002, 0.005, -0.001, 0.002],
    [0.004, -0.001, 0.010, 0.003],
```

```

    [0.000, 0.002, 0.003, 0.007]
])

risk_penalty = 0.5
diversification_weight = 0.05

POP_SIZE = 30
GENERATIONS = 60
MUTATION_RATE = 0.1
CROSSOVER_RATE = 0.8
NUM_ASSETS = 4

def create_individual():
    weights = np.random.rand(NUM_ASSETS)
    return weights / np.sum(weights)

def fitness(weights):
    portfolio_return = np.dot(weights, expected_returns)
    portfolio_risk = np.dot(weights.T, np.dot(cov_matrix, weights))

    diversification = -np.sum(weights**2)

    return portfolio_return - risk_penalty * portfolio_risk + \
        diversification_weight * diversification

def select_parent(population):
    a, b = random.sample(population, 2)
    return a if fitness(a) > fitness(b) else b

def crossover(p1, p2):
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1, NUM_ASSETS - 1)
        child = np.concatenate((p1[:point], p2[point:]))
        return child / np.sum(child)
    return p1

def mutate(individual):
    if random.random() < MUTATION_RATE:
        index = random.randint(0, NUM_ASSETS - 1)
        individual[index] += np.random.uniform(-0.1, 0.1)
        individual = np.maximum(individual, 0)
        return individual / np.sum(individual)
    return individual

population = [create_individual() for _ in range(POP_SIZE)]
best_solution = None

for gen in range(GENERATIONS):

```

```

new_population = []

for _ in range(POP_SIZE):
    p1 = select_parent(population)
    p2 = select_parent(population)
    child = crossover(p1, p2)
    child = mutate(child)
    new_population.append(child)

population = new_population

best = max(population, key=fitness)
if best_solution is None or fitness(best) > fitness(best_solution):
    best_solution = best

print(f"Generation {gen+1}: Best Fitness = {fitness(best_solution):.5f}")

print("\nOptimal Diversified Portfolio:")
for i, w in enumerate(best_solution):
    print(f"Asset {i+1}: {w*100:.2f}%")

print(f"\nExpected Return: {np.dot(best_solution, expected_returns):.4f}")
print(f"Portfolio Risk: {np.dot(best_solution.T, np.dot(cov_matrix, best_solution)):.4f}")
print(f"Final Fitness: {fitness(best_solution):.4f}")

```

Output:

```
Generation 40: Best Fitness = 0.11748
Generation 41: Best Fitness = 0.11748
Generation 42: Best Fitness = 0.11748
Generation 43: Best Fitness = 0.11748
Generation 44: Best Fitness = 0.11748
Generation 45: Best Fitness = 0.11748
Generation 46: Best Fitness = 0.11748
Generation 47: Best Fitness = 0.11748
Generation 48: Best Fitness = 0.11748
Generation 49: Best Fitness = 0.11748
Generation 50: Best Fitness = 0.11748
Generation 51: Best Fitness = 0.11748
Generation 52: Best Fitness = 0.11748
Generation 53: Best Fitness = 0.11748
Generation 54: Best Fitness = 0.11748
Generation 55: Best Fitness = 0.11748
Generation 56: Best Fitness = 0.11748
Generation 57: Best Fitness = 0.11748
Generation 58: Best Fitness = 0.11748
Generation 59: Best Fitness = 0.11748
Generation 60: Best Fitness = 0.11748
```

Optimal Diversified Portfolio:

```
Asset 1: 38.39%
Asset 2: 13.20%
Asset 3: 57.01%
Asset 4: 0.00%
```

```
Expected Return: 0.1448
Portfolio Risk: 0.0056
Final Fitness: 0.1175
```

Program 2

Optimize the green-light durations for the north-south and east-west directions at a four-way intersection so that total vehicle delay is minimized. The task is to determine the best allocation of green-time within a fixed cycle length, given traffic arrival rates for both directions, in order to achieve the lowest combined delay for all vehicles.

Algorithm:

DATE: _____

Lab - 2
Particle Swarm

* Pseudocode

Initialize parameters (num_particles, inertia, cognitive, social, iterations)
Initialize particles with random positions and velocities
Evaluate fitness for each particle
Set personal_best = position
Set global_best = best personal_best
Set personal_best_fitness = fitness

For iteration in 1 to max_iterations:
For each particle:
update velocity:
 $velocity = inertia * velocity$

update position: $position += velocity$
Evaluate fitness

If fitness better than personal_best_fitness:
personal_best = position
personal_best_fitness = fitness

If fitness better than global_best_fitness:
global_best = position
global_best_fitness = fitness

return global_best, global_best_fitness

Code:

```
import numpy as np
import random
```

```
arrival_NS = 35
arrival_EW = 20
```

```
CYCLE_TIME = 120
```



```

def traffic_delay(times):
    g_ns, g_ew = times

    g_ns = max(g_ns, 1)
    g_ew = max(g_ew, 1)

    delay_NS = arrival_NS / g_ns
    delay_EW = arrival_EW / g_ew

    return delay_NS + delay_EW

NUM_PARTICLES = 20
ITERATIONS = 50

w_inertia = 0.7
c1 = 1.4
c2 = 1.4

particles = []
velocities = []

for _ in range(NUM_PARTICLES):
    g_ns = random.uniform(20, 100)
    g_ew = CYCLE_TIME - g_ns
    particles.append(np.array([g_ns, g_ew]))

    velocities.append(np.random.uniform(-5, 5, 2))

def fitness(x):
    return -traffic_delay(x)

p_best_positions = particles.copy()
p_best_scores = [fitness(p) for p in particles]

g_best_position = p_best_positions[np.argmax(p_best_scores)]

for it in range(ITERATIONS):
    for i in range(NUM_PARTICLES):
        r1, r2 = random.random(), random.random()

        velocities[i] = (
            w_inertia * velocities[i]
            + c1 * r1 * (p_best_positions[i] - particles[i])
            + c2 * r2 * (g_best_position - particles[i])
        )

        particles[i] = particles[i] + velocities[i]

```

```

g_ns = particles[i][0]
g_ns = max(5, min(g_ns, 115))
g_ew = CYCLE_TIME - g_ns

particles[i] = np.array([g_ns, g_ew])

score = fitness(particles[i])

if score > p_best_scores[i]:
    p_best_scores[i] = score
    p_best_positions[i] = particles[i]

g_best_position = p_best_positions[np.argmax(p_best_scores)]

print(f"Iteration {it+1}: Best Delay = {-max(p_best_scores):.4f}")

best_ns, best_ew = g_best_position
best_delay = -max(p_best_scores)

print("\nOptimized Traffic Signal Timings (PSO):")
print(f"North-South Green Time: {best_ns:.2f} seconds")
print(f"East-West Green Time: {best_ew:.2f} seconds")
print(f"Total Vehicle Delay Score: {best_delay:.4f}")

```

Output:

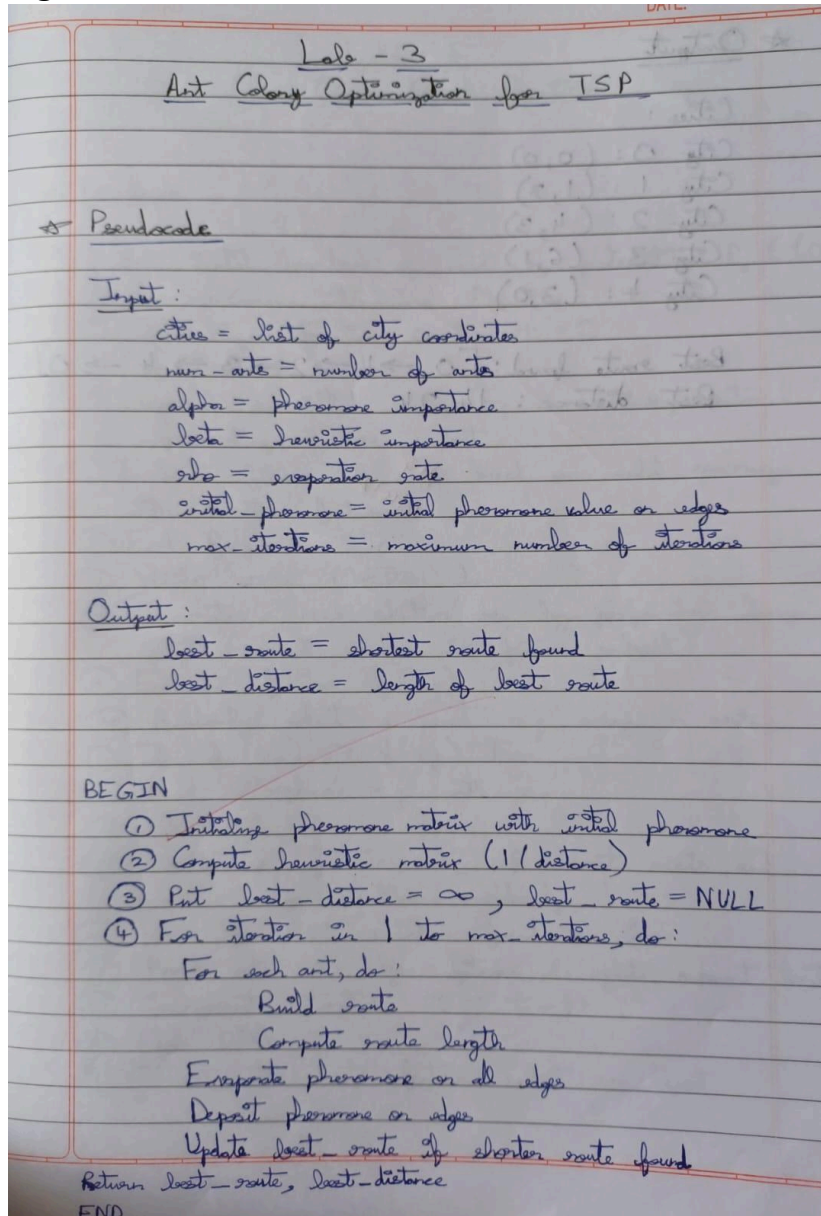
```
Iteration 20: Best Delay = 0.8993
Iteration 21: Best Delay = 0.8993
Iteration 22: Best Delay = 0.8993
Iteration 23: Best Delay = 0.8993
Iteration 24: Best Delay = 0.8993
Iteration 25: Best Delay = 0.8993
Iteration 26: Best Delay = 0.8993
Iteration 27: Best Delay = 0.8993
Iteration 28: Best Delay = 0.8993
Iteration 29: Best Delay = 0.8993
Iteration 30: Best Delay = 0.8993
Iteration 31: Best Delay = 0.8993
Iteration 32: Best Delay = 0.8993
Iteration 33: Best Delay = 0.8993
Iteration 34: Best Delay = 0.8993
Iteration 35: Best Delay = 0.8993
Iteration 36: Best Delay = 0.8993
Iteration 37: Best Delay = 0.8993
Iteration 38: Best Delay = 0.8993
Iteration 39: Best Delay = 0.8993
Iteration 40: Best Delay = 0.8993
Iteration 41: Best Delay = 0.8993
Iteration 42: Best Delay = 0.8993
Iteration 43: Best Delay = 0.8993
Iteration 44: Best Delay = 0.8993
Iteration 45: Best Delay = 0.8993
Iteration 46: Best Delay = 0.8993
Iteration 47: Best Delay = 0.8993
Iteration 48: Best Delay = 0.8993
Iteration 49: Best Delay = 0.8993
Iteration 50: Best Delay = 0.8993
```

```
Optimized Traffic Signal Timings (PS0):
North-South Green Time: 68.34 seconds
East-West Green Time: 51.66 seconds
Total Vehicle Delay Score: 0.8993
```

Program 3

Find the shortest possible delivery route that visits all given cities exactly once and returns to the starting point. The objective is to minimize the total travel distance by determining the most efficient sequence in which the delivery locations should be visited.

Algorithm:



Code:

```
import numpy as np
import random
import math
```

```
cities = np.array([
    [0, 0],
```

```

[2, 4],
[3, 1],
[5, 2],
[6, 6],
[8, 3],
[1, 7],
[4, 5]
])
num_cities = len(cities)

def euclidean(a, b):
    return np.linalg.norm(a - b)

dist_matrix = np.zeros((num_cities, num_cities))
for i in range(num_cities):
    for j in range(num_cities):
        dist_matrix[i][j] = euclidean(cities[i], cities[j])

num_ants = 20
alpha = 1.0
beta = 5.0
rho = 0.5
Q = 100
iterations = 50

pheromone = np.ones((num_cities, num_cities))
heuristic = 1 / (dist_matrix + np.eye(num_cities))

def construct_route(start_city):
    route = [start_city]
    unvisited = set(range(num_cities))
    unvisited.remove(start_city)

    current = start_city

    while unvisited:
        probabilities = []
        for next_city in unvisited:
            tau = pheromone[current][next_city] ** alpha
            eta = heuristic[current][next_city] ** beta
            probabilities.append(tau * eta)

        probabilities = np.array(probabilities)
        probabilities /= probabilities.sum()

        next_city = random.choices(list(unvisited), weights=probabilities)[0]
        route.append(next_city)
        unvisited.remove(next_city)

```

```

        current = next_city

    return route

def route_length(route):
    total = 0
    for i in range(len(route)):
        total += dist_matrix[route[i]][route[(i+1) % num_cities]]
    return total

best_route = None
best_length = math.inf

for it in range(iterations):
    routes = []
    lengths = []

    for ant in range(num_ants):
        start = random.randint(0, num_cities - 1)
        route = construct_route(start)
        L = route_length(route)
        routes.append(route)
        lengths.append(L)

        if L < best_length:
            best_length = L
            best_route = route

    pheromone *= (1 - rho)

    for route, L in zip(routes, lengths):
        for i in range(num_cities):
            a = route[i]
            b = route[(i + 1) % num_cities]
            pheromone[a][b] += Q / L
            pheromone[b][a] += Q / L

    print(f'Iteration {it+1}: Best Length = {best_length:.3f}')

print("\nOptimal Delivery Route (ACO):")
print("-> ".join(str(city) for city in best_route))
print(f'Shortest Distance: {best_length:.3f}')

```

Output:

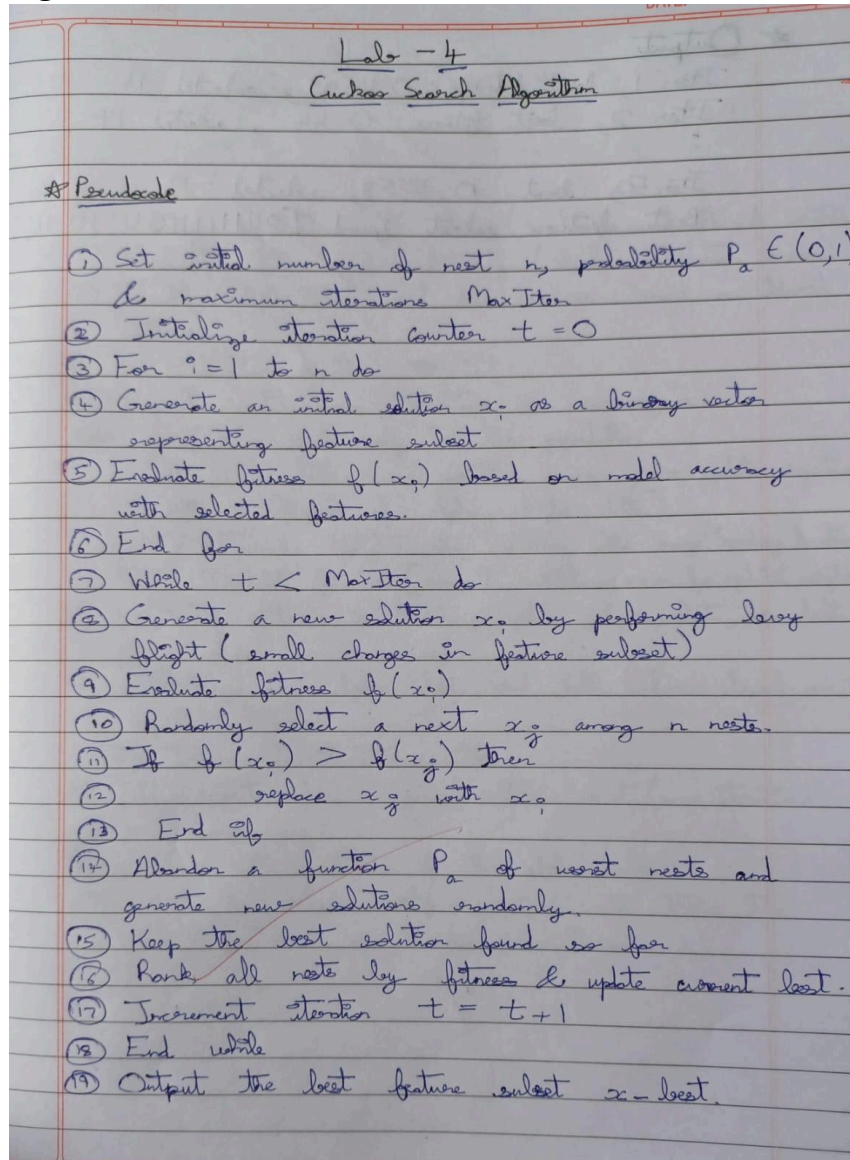
```
Iteration 19: Best Length = 25.642
Iteration 20: Best Length = 25.642
Iteration 21: Best Length = 25.642
Iteration 22: Best Length = 25.642
Iteration 23: Best Length = 25.642
Iteration 24: Best Length = 25.642
Iteration 25: Best Length = 25.642
Iteration 26: Best Length = 25.642
Iteration 27: Best Length = 25.642
Iteration 28: Best Length = 25.642
Iteration 29: Best Length = 25.642
Iteration 30: Best Length = 25.642
Iteration 31: Best Length = 25.642
Iteration 32: Best Length = 25.642
Iteration 33: Best Length = 25.642
Iteration 34: Best Length = 25.642
Iteration 35: Best Length = 25.642
Iteration 36: Best Length = 25.642
Iteration 37: Best Length = 25.642
Iteration 38: Best Length = 25.642
Iteration 39: Best Length = 25.642
Iteration 40: Best Length = 25.642
Iteration 41: Best Length = 25.642
Iteration 42: Best Length = 25.642
Iteration 43: Best Length = 25.642
Iteration 44: Best Length = 25.642
Iteration 45: Best Length = 25.642
Iteration 46: Best Length = 25.642
Iteration 47: Best Length = 25.642
Iteration 48: Best Length = 25.642
Iteration 49: Best Length = 25.642
Iteration 50: Best Length = 25.642

Optimal Delivery Route (AC0):
4 -> 5 -> 3 -> 2 -> 0 -> 1 -> 6 -> 7
Shortest Distance: 25.642
```

Program 4

Determine an optimal drone flight path from a start point to a goal location while avoiding obstacles. The objective is to select a set of waypoints that minimize total travel distance, reduce abrupt directional changes, and prevent collisions, resulting in a smooth and safe trajectory for the drone.

Algorithm:



Code:

```
import numpy as np
import random
```

```
START = np.array([0, 0])
```



```

GOAL = np.array([10, 10])

obstacles = [
    (4, 4, 2),
    (7, 6, 1.5)
]

NUM_WAYPOINTS = 5
NUM_NESTS = 20
PA = 0.25
ITERATIONS = 60
BOUND_LOW, BOUND_HIGH = 0, 10

def random_path():
    return np.random.uniform(BOUND_LOW, BOUND_HIGH, (NUM_WAYPOINTS, 2))

def collision_penalty(path):
    penalty = 0
    for (cx, cy, r) in obstacles:
        for (x, y) in path:
            if np.linalg.norm([x - cx, y - cy]) < r:
                penalty += 1
    return penalty

def path_length(path):
    points = [START] + list(path) + [GOAL]
    dist = 0
    for i in range(len(points) - 1):
        dist += np.linalg.norm(points[i+1] - points[i])
    return dist

def smoothness(path):
    points = [START] + list(path) + [GOAL]
    angle_penalty = 0
    for i in range(1, len(points)-1):
        v1 = points[i] - points[i-1]
        v2 = points[i+1] - points[i]
        cos_angle = np.dot(v1, v2) / (np.linalg.norm(v1)*np.linalg.norm(v2) + 1e-6)
        angle_penalty += (1 - cos_angle)
    return angle_penalty

def fitness(path):
    L = path_length(path)
    C = collision_penalty(path)
    S = smoothness(path)
    return L + 20*C + 0.1*S

def levy_flight(Lambda):

```

```

u = np.random.normal(0, 1)
v = np.random.normal(0, 1)
return u / (abs(v)**(1/Lambda))

nests = [random_path() for _ in range(NUM_NESTS)]
best_path = None
best_cost = float("inf")

for it in range(ITERATIONS):
    new_nests = []
    for nest in nests:
        step = levy_flight(1.5)
        new_path = nest + step * np.random.randn(NUM_WAYPOINTS, 2)
        new_path = np.clip(new_path, BOUND_LOW, BOUND_HIGH)
        new_nests.append(new_path)

    for i in range(NUM_NESTS):
        if fitness(new_nests[i]) < fitness(nests[i]):
            nests[i] = new_nests[i]

    worst_k = int(PA * NUM_NESTS)
    worst_indices = np.argsort([fitness(n) for n in nests])[-worst_k:]

    for idx in worst_indices:
        nests[idx] = random_path()

    current_best_idx = np.argmin([fitness(n) for n in nests])
    current_cost = fitness(nests[current_best_idx])

    if current_cost < best_cost:
        best_cost = current_cost
        best_path = nests[current_best_idx]

    print(f'Iteration {it+1}: Best Cost = {best_cost:.4f}')

print("\nOptimal Drone Path Found (Cuckoo Search):")
print("Waypoints:")
print(best_path)
print(f'\nFinal Cost = {best_cost:.4f}')

```

Output:

```
Iteration 35: Best Cost = 16.8962
Iteration 36: Best Cost = 16.8962
Iteration 37: Best Cost = 16.8962
Iteration 38: Best Cost = 16.8962
Iteration 39: Best Cost = 16.8962
Iteration 40: Best Cost = 15.2999
Iteration 41: Best Cost = 15.2999
Iteration 42: Best Cost = 15.2999
Iteration 43: Best Cost = 15.2999
Iteration 44: Best Cost = 15.2999
Iteration 45: Best Cost = 15.2999
Iteration 46: Best Cost = 15.2999
Iteration 47: Best Cost = 15.2999
Iteration 48: Best Cost = 15.2999
Iteration 49: Best Cost = 15.2999
Iteration 50: Best Cost = 15.2999
Iteration 51: Best Cost = 15.2999
Iteration 52: Best Cost = 15.2999
Iteration 53: Best Cost = 15.2999
Iteration 54: Best Cost = 15.2999
Iteration 55: Best Cost = 15.2999
Iteration 56: Best Cost = 15.2999
Iteration 57: Best Cost = 15.2999
Iteration 58: Best Cost = 15.2999
Iteration 59: Best Cost = 15.2999
Iteration 60: Best Cost = 15.2999
```

Optimal Drone Path Found (Cuckoo Search):

Waypoints:

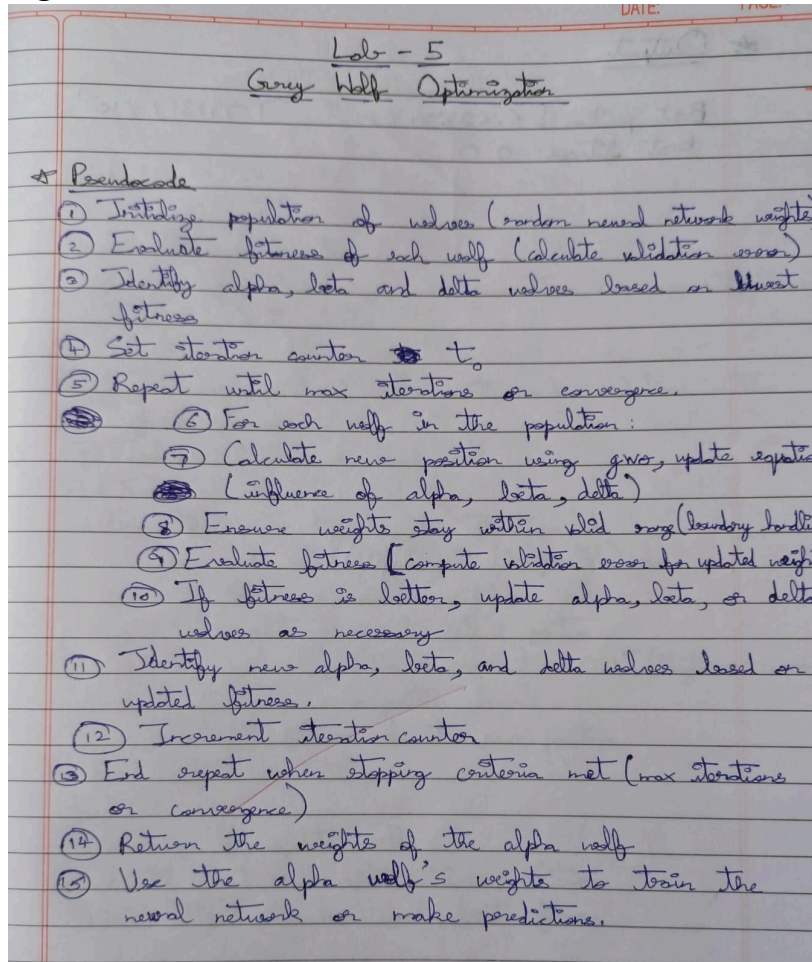
```
[[1.38706259 4.35472108]
 [1.52410494 4.33870281]
 [7.41759554 8.12580394]
 [8.57886601 9.15409272]
 [9.49373736 9.19667086]]
```

Final Cost = 15.2999

Program 5

Identify the best hyperparameters for a Support Vector Machine classifier on the Iris dataset. The goal is to find values of C and gamma that maximize cross-validated classification accuracy within specified parameter bounds.

Algorithm:



Code:

```
import numpy as np
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score
```

```
iris = datasets.load_iris()
X = iris.data
y = iris.target
```

```

def fitness(params):
    C, gamma = params
    C = max(C, 1e-5)
    gamma = max(gamma, 1e-5)
    model = SVC(C=C, gamma=gamma)
    score = cross_val_score(model, X, y, cv=5).mean()
    return -score

def GWO(fitness, dim, n_wolves=10, n_iter=30, lb=None, ub=None):
    if lb is None:
        lb = np.zeros(dim)
    if ub is None:
        ub = np.ones(dim)

    wolves = np.random.uniform(lb, ub, (n_wolves, dim))

    alpha_pos, alpha_score = None, float("inf")
    beta_pos, beta_score = None, float("inf")
    delta_pos, delta_score = None, float("inf")

    for t in range(n_iter):
        a = 2 - t * (2 / n_iter)

        for i in range(n_wolves):
            wolves[i] = np.clip(wolves[i], lb, ub)

            score = fitness(wolves[i])

            if score < alpha_score:
                alpha_score, alpha_pos = score, wolves[i].copy()
            elif score < beta_score:
                beta_score, beta_pos = score, wolves[i].copy()
            elif score < delta_score:
                delta_score, delta_pos = score, wolves[i].copy()

        for i in range(n_wolves):
            r1, r2 = np.random.rand(dim), np.random.rand(dim)
            A1 = 2 * a * r1 - a
            C1 = 2 * r2
            D_alpha = abs(C1 * alpha_pos - wolves[i])
            X1 = alpha_pos - A1 * D_alpha

            r1, r2 = np.random.rand(dim), np.random.rand(dim)
            A2 = 2 * a * r1 - a
            C2 = 2 * r2
            D_beta = abs(C2 * beta_pos - wolves[i])
            X2 = beta_pos - A2 * D_beta

```

```

r1, r2 = np.random.rand(dim), np.random.rand(dim)
A3 = 2 * a * r1 - a
C3 = 2 * r2
D_delta = abs(C3 * delta_pos - wolves[i])
X3 = delta_pos - A3 * D_delta

wolves[i] = (X1 + X2 + X3) / 3

print(f'Iteration {t+1}: Best Fitness = {-alpha_score:.5f}')

return alpha_pos, -alpha_score

lb = [0.01, 0.001]
ub = [100, 1]

best_params, best_score = GWO(fitness, dim=2, n_wolves=15, n_iter=30, lb=lb, ub=ub)

print("\nOptimal SVM Parameters Found:")
print(f'C = {best_params[0]:.5f}, gamma = {best_params[1]:.5f}')
print(f'Cross-Validated Accuracy = {best_score:.5f}')

```

Output:

```
Iteration 1: Best Fitness = 0.98000
Iteration 2: Best Fitness = 0.98000
Iteration 3: Best Fitness = 0.98000
Iteration 4: Best Fitness = 0.98667
Iteration 5: Best Fitness = 0.98667
Iteration 6: Best Fitness = 0.98667
Iteration 7: Best Fitness = 0.98667
Iteration 8: Best Fitness = 0.98667
Iteration 9: Best Fitness = 0.98667
Iteration 10: Best Fitness = 0.98667
Iteration 11: Best Fitness = 0.98667
Iteration 12: Best Fitness = 0.98667
Iteration 13: Best Fitness = 0.98667
Iteration 14: Best Fitness = 0.98667
Iteration 15: Best Fitness = 0.98667
Iteration 16: Best Fitness = 0.98667
Iteration 17: Best Fitness = 0.98667
Iteration 18: Best Fitness = 0.98667
Iteration 19: Best Fitness = 0.98667
Iteration 20: Best Fitness = 0.98667
Iteration 21: Best Fitness = 0.98667
Iteration 22: Best Fitness = 0.98667
Iteration 23: Best Fitness = 0.98667
Iteration 24: Best Fitness = 0.98667
Iteration 25: Best Fitness = 0.98667
Iteration 26: Best Fitness = 0.98667
Iteration 27: Best Fitness = 0.98667
Iteration 28: Best Fitness = 0.98667
Iteration 29: Best Fitness = 0.98667
Iteration 30: Best Fitness = 0.98667
```

```
Optimal SVM Parameters Found:
C = 79.69518, gamma = 0.00332
Cross-Validated Accuracy = 0.98667
```

Program 6

Optimize the assignment of tasks across multiple processor cores so that the overall makespan—the maximum load on any core—is minimized. Each task has a known processing time, and the objective is to find a distribution of tasks to cores that achieves the most balanced workload across the system.

Algorithm:

DATE: PAGE:

Lab - 6
Parallel Cellular Algorithm

* Pseudocode

Parallel Cellular Algorithm:

Input: population_size, max_generations, crossover_rate, mutation_rate

Output: best_solution

- ① Initialize grid of cells with random solutions within given bounds
- ② Evaluate fitness of each cell
- ③ // Evolution Loop:
For iteration = 1 to Max Iterations do
for each cell in the grid (in parallel) do
Select a random neighbour cell
update current cell's position:
$$\text{new_position} = \text{current} + 0.1 * (\text{Neighbour} - \text{current})$$

Clip new-position bounds
Evaluate fitness of the new position
End For

Synchronize all updated cells
Find the best cell with minimum fitness
Print Iteration Numbers and best fitness

END FOR

Output the best solution and its fitness value
END

Code:

```
import numpy as np
```

N = 10


```

M = 3
tasks = np.random.randint(1, 20, size=N)

grid_size = (5, 5)
iterations = 50

cells = np.random.randint(0, M, size=(grid_size[0], grid_size[1], N))

def fitness(cell):
    core_times = np.zeros(M)
    for task_idx, core in enumerate(cell):
        core_times[core] += tasks[task_idx]
    return np.max(core_times)

def get_neighbors(grid, i, j):
    neighbors = []
    for di, dj in [(-1,0), (1,0), (0,-1), (0,1)]:
        ni, nj = i+di, j+dj
        if 0 <= ni < grid_size[0] and 0 <= nj < grid_size[1]:
            neighbors.append(grid[ni, nj])
    return neighbors

best_solution = None
best_fitness = float('inf')

for it in range(iterations):
    new_cells = cells.copy()
    for i in range(grid_size[0]):
        for j in range(grid_size[1]):
            neighbors = get_neighbors(cells, i, j)
            neighbor_fitness = [fitness(n) for n in neighbors]
            if neighbors and min(neighbor_fitness) < fitness(cells[i, j]):
                best_neighbor = neighbors[np.argmin(neighbor_fitness)]
                new_cells[i, j] = best_neighbor.copy()
                if np.random.rand() < 0.2:
                    task_to_change = np.random.randint(0, N)
                    new_cells[i, j, task_to_change] = np.random.randint(0, M)
    cells = new_cells

    for i in range(grid_size[0]):
        for j in range(grid_size[1]):
            f = fitness(cells[i, j])
            if f < best_fitness:
                best_fitness = f
                best_solution = cells[i, j].copy()

```

```
print(f'Iteration {it+1}: Best Makespan = {best_fitness}')

print("\nOptimal Task Assignment:")
for task_idx, core in enumerate(best_solution):
    print(f'Task {task_idx+1} -> Core {core+1}')
print(f'Minimum Makespan: {best_fitness}')
```

Output:

```
Iteration 28: Best Makespan = 40.0
Iteration 29: Best Makespan = 40.0
Iteration 30: Best Makespan = 40.0
Iteration 31: Best Makespan = 40.0
Iteration 32: Best Makespan = 40.0
Iteration 33: Best Makespan = 40.0
Iteration 34: Best Makespan = 40.0
Iteration 35: Best Makespan = 40.0
Iteration 36: Best Makespan = 40.0
Iteration 37: Best Makespan = 40.0
Iteration 38: Best Makespan = 40.0
Iteration 39: Best Makespan = 40.0
Iteration 40: Best Makespan = 40.0
Iteration 41: Best Makespan = 40.0
Iteration 42: Best Makespan = 40.0
Iteration 43: Best Makespan = 40.0
Iteration 44: Best Makespan = 40.0
Iteration 45: Best Makespan = 40.0
Iteration 46: Best Makespan = 40.0
Iteration 47: Best Makespan = 40.0
Iteration 48: Best Makespan = 40.0
Iteration 49: Best Makespan = 40.0
Iteration 50: Best Makespan = 40.0
```

Optimal Task Assignment:

```
Task 1 -> Core 1
Task 2 -> Core 2
Task 3 -> Core 1
Task 4 -> Core 2
Task 5 -> Core 1
Task 6 -> Core 3
Task 7 -> Core 1
Task 8 -> Core 1
Task 9 -> Core 3
Task 10 -> Core 3
Minimum Makespan: 40.0
```

Program 7

Select the most informative subset of features from a breast cancer dataset to maximize the classification accuracy of a machine learning model. The objective is to determine which features should be included so that the resulting feature subset yields the highest cross-validated performance while avoiding unnecessary or irrelevant inputs.

Algorithm:

DATE: PAGE:

Lab - 7
Optimization via Gene Expression Algorithm

Step 1: fitness function: $f(x) = x^2$
Encoding technique: 0 to 31
use chromosome of fixed length (genotype)

Step 2: Initial population

SN	Initial chromosome (genotype)	Phenotype (expression)	value	fitness	P
1	+xx	x^2	12	144	0.1247
2	+xx	$2x$	25	625	0.5411
3	x	x	5	25	0.0216
4	-x2	$x-2$	19	361	0.3125

Sum: 1155
Avg: 288.75
Max: 625

Actual Count	Expected Count
1	0.5
2	2.1
0	0.08
1	0.8

Code:

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier
```

```
data = load_breast_cancer()
```

```

X = data.data
y = data.target
num_features = X.shape[1]

population_size = 20
generations = 30
mutation_rate = 0.1
crossover_rate = 0.8
num_genes = num_features

population = np.random.randint(2, size=(population_size, num_genes))

def fitness(individual):
    selected_features = [i for i, gene in enumerate(individual) if gene == 1]
    if not selected_features:
        return 0
    X_subset = X[:, selected_features]
    clf = RandomForestClassifier(n_estimators=50, random_state=42)
    score = cross_val_score(clf, X_subset, y, cv=3).mean()
    return score

def select(population, fitness_vals):
    probs = fitness_vals / np.sum(fitness_vals)
    idx = np.random.choice(len(population), size=2, p=probs)
    return population[idx[0]], population[idx[1]]

def crossover(parent1, parent2):
    if np.random.rand() < crossover_rate:
        point = np.random.randint(1, num_genes-1)
        child1 = np.concatenate([parent1[:point], parent2[point:]])
        child2 = np.concatenate([parent2[:point], parent1[point:]])
        return child1, child2
    return parent1.copy(), parent2.copy()

def mutate(individual):
    for i in range(num_genes):
        if np.random.rand() < mutation_rate:
            individual[i] = 1 - individual[i]
    return individual

best_solution = None
best_fitness = 0

for gen in range(generations):
    fitness_vals = np.array([fitness(ind) for ind in population])

    max_idx = np.argmax(fitness_vals)
    if fitness_vals[max_idx] > best_fitness:

```

```

best_fitness = fitness_vals[max_idx]
best_solution = population[max_idx].copy()

new_population = []
while len(new_population) < population_size:
    p1, p2 = select(population, fitness_vals)
    c1, c2 = crossover(p1, p2)
    c1 = mutate(c1)
    c2 = mutate(c2)
    new_population.extend([c1, c2])

population = np.array(new_population[:population_size])
print(f"Generation {gen+1}: Best Fitness = {best_fitness:.4f}")

selected_features = [i for i, gene in enumerate(best_solution) if gene == 1]
print("\nOptimal Feature Subset Indices:", selected_features)
print("Number of Features Selected:", len(selected_features))
print("Best Cross-Validated Accuracy:", best_fitness)

```

Output:

```
Generation 1: Best Fitness = 0.9631
Generation 2: Best Fitness = 0.9649
Generation 3: Best Fitness = 0.9649
Generation 4: Best Fitness = 0.9666
Generation 5: Best Fitness = 0.9666
Generation 6: Best Fitness = 0.9684
Generation 7: Best Fitness = 0.9684
Generation 8: Best Fitness = 0.9684
Generation 9: Best Fitness = 0.9684
Generation 10: Best Fitness = 0.9684
Generation 11: Best Fitness = 0.9684
Generation 12: Best Fitness = 0.9684
Generation 13: Best Fitness = 0.9684
Generation 14: Best Fitness = 0.9684
Generation 15: Best Fitness = 0.9684
Generation 16: Best Fitness = 0.9684
Generation 17: Best Fitness = 0.9684
Generation 18: Best Fitness = 0.9684
Generation 19: Best Fitness = 0.9684
Generation 20: Best Fitness = 0.9684
Generation 21: Best Fitness = 0.9684
Generation 22: Best Fitness = 0.9684
Generation 23: Best Fitness = 0.9684
Generation 24: Best Fitness = 0.9684
Generation 25: Best Fitness = 0.9684
Generation 26: Best Fitness = 0.9684
Generation 27: Best Fitness = 0.9684
Generation 28: Best Fitness = 0.9684
Generation 29: Best Fitness = 0.9684
Generation 30: Best Fitness = 0.9684

Optimal Feature Subset Indices: [0, 1, 2, 7, 8, 10, 11, 16, 17, 21, 23, 24, 28]
Number of Features Selected: 13
Best Cross-Validated Accuracy: 0.9683746403044649
```